
Project2

请在 3 月 30 日之前于学在浙大提交

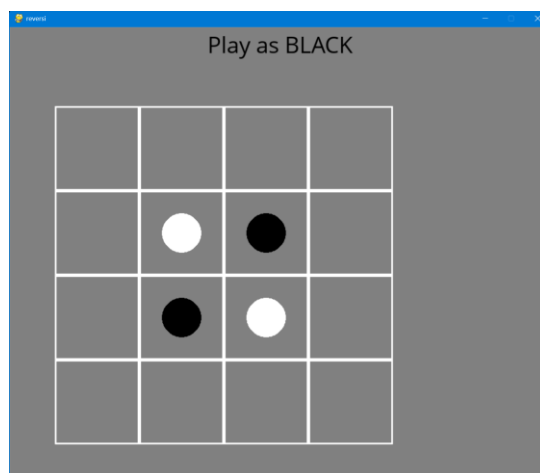
一、实验目标

经过课堂上的学习和课后的复习，相信大家对对抗搜索问题和方法已经有了一定的了解。为了进一步巩固学习成果，我们将分别使用这些搜索方法（Minimax、AlphaBeta 剪枝和 MCTS）写一个简单的游戏 AI。

二、题目描述

本次作业，我们将实现一个简化版的翻转棋——棋盘大小为 4x4、6x6 或 8x8 且仅考虑水平和竖直方向上的翻转。具体规则如下：

1. 黑方先手下，两者轮流落子，且开局如下图所示；



2. 落子方能够落子的位置，要求其为空且落子后能引起翻转，即在水平或竖直方向上存在己方的一个棋子使得棋子到该位置之间为连续的一个及以上的对方棋子；

3. 在落子后，在水平和竖直方向上，若存在己方的棋子满足该棋子到落子的位置之间均为对方棋子，则将这些对方棋子翻转成己方棋子；

4. 若在对局中，落子方不存在能够落子的位置，则轮空；

5. 若双方均轮空，则游戏结束；

6. 游戏结束后，根据棋盘上的子数多少判断胜负，子数多的一方为胜，若子数一样多则为平局。

三、作业内容

在 src 文件夹的 agent.py 和 utils.py 中，给出了包含 Minimax、Alphabeta 剪枝和 MCTS 这 3 种对抗搜索算法代码框架和游戏规则的代码，请你将空缺部分补充完整。

需补充的地方有 TODO 和 NOTE 的提示（一共有 10 处）。

建议补充顺序：Board 类 (utils.py) → Minimax 算法 (agent.py) → Alphabeta 剪枝 (agent.py) → Record 类 (utils.py) → MCTS (agent.py)，按照“TODO”标记的顺序。

在完成 Minimax 算法后，打开终端，在 src 目录下输入“python runner.py --ai_method minimax --board_size 4”即可与使用 minimax 算法的 AI 于 4x4 的棋盘上对战。**注意：6x6 以及 8x8 棋盘在无剪枝情况下分支太多，minimax 算法运行极慢！**

若要使用 Alphabeta 剪枝或 MCTS 版本的 AI，则可将“--ai_method”后的参数更改为“alphabeta”或“ucbl”。特别地，对于 MCTS 算法，我们可以通过“--max_iter”参数指定模拟次数。**注意：对于 alphabeta 剪枝算法，在 6x6 棋盘上使用白棋的第一步耗时约为 13 秒，使用黑棋建议第一步下在左上的白棋上方，耗时约为 14 秒。**

在作业开始之前，请安装“requirements.txt”中提供的 python 包，建议利用 conda 新建环境“conda create -n reversi python==3.12”，“conda activate reversi”后可通过“pip install -r requirements.txt”完成。

四、提交要求

请提交一个以 proj2_学号.zip 命名的 zip 格式压缩文件，此压缩文件应当包含：

- 1) 整个 src 文件夹，包括补充完整的 utils.py、agent.py 和其他 py 文件；（50）
- 2) 作业报告 report.pdf。（50）

注意，请在你补充的代码中必要的地方写上注释。

作业报告要求：

- 1) 对上述 3 个对抗搜索算法进行简要的比较和说明，了解对应的算法性质（最优性、复杂度等）；（30）
- 2) 给出一个游戏进行中 alpha-beta 剪枝的实例：使用手绘、PPT 或者其他工具绘制某一步的博弈树，标注出具体是如何剪枝的；（10）
- 3) 在 MCTS 算法中，改变模拟次数，算法的运行结果有何变化？（10）
- 4) 报告的具体格式没有要求。